

Received 18 October 2024, accepted 11 November 2024, date of publication 15 November 2024,
date of current version 25 November 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3498957

RESEARCH ARTICLE

Recursive Sparse Identification of Nonlinear Dynamics for Online Equation Discovery

SWADEXI ISTIQPHARA^{ID 1,2}, (Graduate Student Member, IEEE),
OYAS WAHYUNGGORO^{ID 1}, (Member, IEEE), AND ADHA IMAM CAHYADI^{ID 1}

¹Department of Electrical and Information Engineering, Universitas Gadjah Mada, Yogyakarta 55281, Indonesia

²Department of Electrical Engineering, Institut Teknologi Sumatera, Lampung 35365, Indonesia

Corresponding author: Oyas Wahyunggoro (oyas@ugm.ac.id)

This work was supported in part by Universitas Gadjah Mada through the IEEE Token for Article Processing Charges (APC), in part by the Center for Higher Education Funding (BPPT), and in part by the Indonesia Endowment Fund for Education (LPDP).

ABSTRACT The dynamic model equations are essential in system analysis and control system design. In adaptive control systems, the mathematical equations of the controlled system are utilized to compute the corresponding control signals based on the current dynamic conditions of the system. The challenge intensifies when dealing with nonlinear systems, as the equation discovery process becomes more intricate. This paper proposes the Recursive Sparse Identification of Nonlinear Dynamics (R-SINDy) using Least Square-Assisted LASSO (Least Absolute Shrinkage and Selection Operator), an extension of the previous SINDy method that processed data in batches for online equation discovery. This method aims to generate mathematical equations for both linear and nonlinear dynamic systems in online streaming data. The proposed method is tested on two nonlinear systems: Lorenz Chaotic System with parameter changes and dataset of KUKA robotic manipulator. The results indicate that the proposed method has the ability to quickly adjust the gain when changes occur because of the adjustment in the forgetting factor, achieving an accuracy of up to 100 % on the Lorenz system and 92.02 % on the robotic manipulator, with a sparsity coefficient of up to 87.59 % from a total of 282 available matrix coefficients.

INDEX TERMS Data driven modeling, nonlinear dynamics, online system identification, recursive SINDy, sparse identification, system modeling, streaming data.

I. INTRODUCTION

The dynamic system model is a mathematical representation used to depict the dynamic behavior exhibited by a system. With the presence of a dynamic model, the characteristics of a system can be analyzed, and the design of the control system can be facilitated to achieve optimal outcomes [1], [2]. The dynamics of a system can be categorized into two types based on linearity [3]: linear systems, in which the output is proportional to the input and nonlinear systems, in which the output is not proportional to the input. In general, the dynamics of a system are fundamentally nonlinear primarily because of the presence of several environmental

variables that influence the system. However, some systems exhibit low nonlinearity, allowing nonlinear variables to be neglected, and the system is then considered as a linear system. Moreover, Nonlinear systems can be linearized for control design purposes [4], [5]. Although the system can be considered linear, nonlinearities can still emerge and play a significant role in certain cases, potentially leading to control-system failures. Therefore, it is crucial to remain attentive to the nonlinear nature of a system to ensure the development of a robust control system design.

Two techniques can be employed to derive the dynamic equations of a system: manual identification and computer-based system identification. In manual system identification, the system variable is measured, and Newton's law analysis for the mechanical system or Ohm's law analysis for the

The associate editor coordinating the review of this manuscript and approving it for publication was Hassen Ouakad^{ID}.

electric circuit system is applied to understand the influence of each component, which are then modeled into mathematical equations [6]. Computer-based system identification [7] can be categorized into two approaches: parameter estimation (grey box modeling) and black box modeling. Parameter estimation utilizes the system equation structure obtained manually, and its parameters are identified using a computer through the measurement data of the system's input and output [8], [9], [10]. Black-box modeling, involves modeling a system as an unknown structure without the knowledge of its mathematical equations. To implement this technique, measurement data of the system input and output are required, which are then processed using a computer.

Some studies related to black-box modeling include the use of machine learning methods such as artificial neural networks and regression methods. On system identification based on artificial neural networks, the generated model retains the characteristics of a universal approximator [11], which, on one hand, offers high flexibility in representing various types of mathematical functions. However, the inherent complexity of the Universal Approximator may pose challenges for further analysis of the model structure. One of the advantages of deep learning based system identification is its ability to model various complex phenomena due to the large number of layers in the network. However, one drawback of deep learning is its high computational cost [12], although some studies have been conducted to reduce this cost and improve computational efficiency [13].

Recent studies on system identification using a regression technique known as Sparse Identification of Nonlinear Dynamics (SINDy) have demonstrated its capability to derive mathematical equations for nonlinear systems [14], [15]. The method employs the sequentially thresholded least-squares (STLS) regression approach to fit the input and output data by learning the active library components. Notably, it has undergone extensive advancements, incorporating the integration of a model selection criterion known as SINDy with Akaike Information Criteria (AIC) [16]. These criteria facilitate the selection of the optimal model from various candidates, each associated with a distinct threshold value. The SINDy-AIC method effectively addresses the limitations of the traditional SINDy, which is constrained to a single threshold. Research on SINDy for low-data limits was also explored in [17], known as Ensemble-SINDy, utilizing a dataset with a modest size of only 100 entries. SINDy-MPC [18], which also employs STLS, extends the SINDy method by integrating it with Model Predictive Control (MPC) to forecast the system's behavior over multiple future horizons. This method can predict up to ten data horizons, although it still requires batch data for the training process. Among the various methods, it is evident that the least-squares regression method is effective for batch data processing. In the context of single-data processing, the least-squares regression method was further extended to Recursive Least Squares [19].

The Recursive Least Squares (RLS) is a regression method that can replace Ordinary Least Squares (OLS) for processing streaming data. The RLS method stands out for its more efficient computation, utilizing a smaller amount of data than OLS [20]. RLS is often applied in adaptive control, filter, and system identification [21]. RLS was introduced by Gauss in 1821 and resurfaced in 1950 [19], thereby opening up opportunities for real-time applications. RLS still being used and developed, and various RLS variants have been proposed, including RLS with Forgetting Factor and Lattice RLS [22], [23]. RLS is widely used in adaptive control systems [24] and can be applied to both linear and nonlinear systems. However, with existing algorithms, all the resulting gains tend to have values, and has not yet been applied to produce complex mathematical equations. Further development is required for Recursive Least Squares to yield sparse solutions, allowing for better interpretability in the context of dynamic systems.

Many studies have been conducted to develop RLS so that it can be used to process streaming data and produce sparse gain, such as Sparse RLS with L0-Norm Sparse RLS [25], RLS Algorithm with L1 Regularized Least Square (SPARLS) [26], Zero-Attracting Recursive Least Squares Algorithms [27], and Sparse Sliding-Window Kernel Recursive Least-Squares [28] which is good for simple sparse system identification. However, for more complex nonlinear systems, further research is needed to produce a more sparse coefficient matrix for highly coupled systems. Least Absolute Shrinkage and Selection Operator (LASSO) is a regression method that is able to eliminate less important gains, thus producing more sparse gains. In [29], using the LASSO method for the equation discovery of a 3-DoF robot manipulator, the results outperformed sequential threshold least squares (STLS) in producing a sparser coefficient matrix. Given its superior capability in processing batch data, the LASSO method has better potential for use in streaming data processing because it can eliminate less important gains. However, the challenge in implementing recursive LASSO is that an imprecise penalty can result in the elimination of less precise coefficients and result in a slow process to find the right matrix coefficients.

To address these challenges, existing nonlinear system identification methods demand extensive datasets, making them less effective for applications in system monitoring, adaptive control systems or real-time fault detection. These scenarios require the ability to process data continuously and promptly adapt to changes in the system dynamics, and Recursive LASSO has emerged as a solution for online utilization. However, further refinement of Recursive LASSO is necessary to produce sparser equations. In this study, the RLS algorithm assists recursive LASSO in processing streaming data of nonlinear systems by incorporating techniques for input data nonlinearization, parameter change detection, model selection, online sparse identification algorithms, and equation construction. These modifications aim to unveil the sparse coefficient matrix that represent the mathematical

equations of nonlinear systems dynamics and enhance the applicability of the regression technique to streaming data.

The first contribution of this research is the sparse recursive system identification of nonlinear dynamics (R-SINDy) to discover the mathematical equation of the system using streaming of single data continuously. The modification was performed by adding nonlinearization and sparse algorithms.

Second, RLS has better capability in modeling simple systems, whereas recursive LASSO has better capability in complex nonlinear systems. In this study, RLS was proposed to assist recursive LASSO in speeding up the process of finding the right matrix coefficients.

Third, when there is a parameter change in the system that can be caused by a fault or external disturbance, both the RLS and R-LASSO require more time to obtain new and accurate matrix coefficients. A Dynamic Forgetting Factor using parameter change detection is proposed to detect changes in the parameters and accelerate the search for new coefficients.

Fourth, employing only one threshold in Recursive LASSO renders it ineffective and poses several potential issues. This is because of the risk of eliminating numerous or all the coefficients in each iteration, leading to inaccurate equations. To address this challenge, the model selection criteria were used to select a model with an appropriate penalty at each iteration to ensure an accurate model.

Fifth, an online equation discovery technique is proposed to model system dynamics, which is designed to perform effectively not only under ideal conditions but also when confronted with changes in the underlying system dynamics equation, exemplifying its versatility and adaptability in dynamic environments.

The remainder of this paper is organized as follows. Section II discusses the nonlinear systems used in this study, namely, the Lorenz system and the robot manipulator. Section III covers the study's methodology. Section IV presents and discusses the results of this study. Finally, Section V concludes the paper.

II. NONLINEAR SYSTEM

In this study, two known nonlinear system models with dynamic equations are employed to test the effectiveness of the proposed method. The two models are the Lorenz system [14], [30] and 3-DOF robot manipulator [31].

A. LORENZ SYSTEM

An example of a system that has nonlinear characteristics is the Lorenz chaotic system, the dynamic model equation of Lorenz system is expressed as follows [30]:

$$\begin{aligned}\dot{x}_L &= \sigma(y_L - x_L) \\ \dot{y}_L &= x_L(\rho - z_L) - y_L \\ \dot{z}_L &= x_L y_L - \beta z_L\end{aligned}\quad (1)$$

where x_L, y_L , and z_L denote the state of the Lorenz system and σ, ρ , and β denote the system parameters. The Lorenz

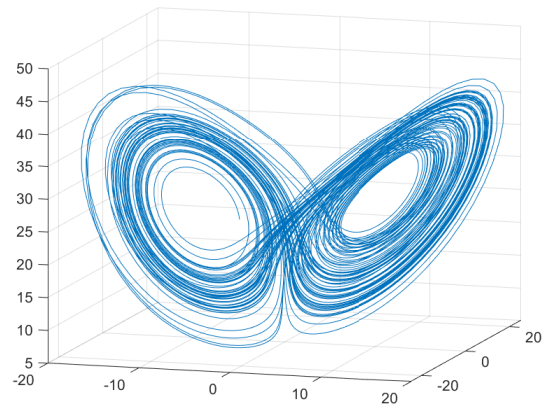


FIGURE 1. Lorenz system.

system exhibited chaotic behavior, as shown in Fig. 1, that is, although the dynamics equations of the system are relatively simple, this system is very sensitive to initial conditions, small changes in a system's initial conditions can produce very different changes in each state over time. Based on these characteristics, the Lorenz system can be considered an appropriate nonlinear system to test the effectiveness of the proposed method.

B. KUKA ROBOT MANIPULATOR

The Kuka robot manipulator is a highly coupled nonlinear system with multiple inputs and multiple outputs (MIMO). The robot has an arm-like structure consisting of three arm links and three joints that serve as its driving force, as illustrated in Fig. 2. The robot used in this study was the KUKA KR 300 2500 ultra-SE industrial robot, with the dataset obtained from [31].

This robot consists of six joints, each of which has two states, namely the velocity and angular position of the joint,



FIGURE 2. Kuka robot manipulator. The permission to use the image has been obtained from [31].

for a total of 12 states. To be used in inverse dynamics, one additional state is required in each joint, namely, the angular acceleration state, which is obtained by deriving the velocity state. During use, this robot experiences backlash, pose-dependent hydraulic forces, and other nonlinear forces that affect the movement of each joint.

In this study, only the first three joints were identified with their respective data; namely, $q_{1,2,3}$ are the angular positions of the joint, $\dot{q}_{1,2,3}$ is the angular velocity of the joint, and $\ddot{q}_{1,2,3}$ is the angular acceleration of the joint.

III. RECURSIVE SPARSE IDENTIFICATION OF NONLINEAR DYNAMICS

This section discusses the proposed method, called Recursive Sparse Identification of Nonlinear Dynamics (R-SINDy). The flowchart of this method is shown in Fig. 3, where R-SINDy consists of five main parts: Nonlinearization, Recursive Least Squares with Forgetting Factor, Recursive LASSO with Adam Optimizer, Parameter Changes Detector and Model Selection.

A. NONLINEARIZATION

To be able to work on nonlinear systems, it is necessary to carry out nonlinearization, which converts input data to nonlinear data through nonlinear functions.

There are two nonlinearization techniques used in this study, namely, a basic technique using the binomial coefficient combination algorithm [14] used in the Lorenz system and the Dynamic Expression Nonlinearization (DEx-N)-Algorithm [29] for nonlinearization in a 3-DOF Robot Manipulator. The difference between these techniques lies in the use of all variables. In the Lorenz system, nonlinearization is not limited by rules for using variables, whereas in the robot manipulator system, for nonlinearization to be more effective, rules need to be used so that only certain variables are nonlinearized.

B. RECURSIVE LEAST SQUARE WITH FORGETTING FACTOR

The objective function of system modeling aims to minimize the difference between the estimated output and the actual output, as stated in the following equation:

$$\min_{\Xi} \|X\Xi - Y\|_2^2 \quad (2)$$

where $X \in \mathbb{R}^{m \times n}$ is the system input, $Y \in \mathbb{R}^m$ is the system output, and $\Xi \in \mathbb{R}^n$ is the gain coefficient that is optimized to minimize the objective function. The gain coefficient Ξ is obtained using the following ordinary least squares equation, as also applied in the SINDy framework [14]. First, the objective function is expanded as follows:

$$\|Y - X\Xi\|_2^2 = Y^T Y - 2Y^T X\Xi + \Xi^T X^T X\Xi \quad (3)$$

Take the gradient of (3) and set it equal to zero:

$$\frac{\partial}{\partial \Xi} (Y^T Y - 2Y^T X\Xi + \Xi^T X^T X\Xi) = 0 \quad (4)$$

The derivative with respect to Ξ is:

$$-2X^T Y + 2X^T X\Xi = 0 \quad (5)$$

Simplifying this equation:

$$X^T X\Xi = X^T Y \quad (6)$$

Finally, solving for Ξ , the ordinary least squares solution is obtained as follows:

$$\Xi = (X^T X)^{-1} X^T Y \quad (7)$$

Based on the Least Square equation in (7), as the value of X increases, the computational process becomes increasingly complex, inefficient, and expensive. By utilizing the Sherman-Morrison formula and matrix inversion lemma, the Recursive Least Squares (RLS) algorithm can be expressed as follows [20], [32], and [33]:

The modeling error E_k at iteration k is defined as:

$$E_k = y_k - \Xi_k^T x_k \quad (8)$$

where y_k denotes the output of the system, $\Xi_k^T \in \mathbb{R}^m$ is the coefficient matrix, and $x_k \in \mathbb{R}^m$ is the input vector. β is calculated as:

$$\beta = A_{k-1} x_k \quad (9)$$

where A_{k-1} denotes the covariance matrix. Then, α which can be called the dynamic learning gain, is determined using the following equation:

$$\alpha = (\lambda + x^T \beta)^{-1} \quad (10)$$

where λ is the forgetting factor. The covariance matrix P is updated as follows:

$$A = \frac{1}{\lambda} (A - \alpha \beta \beta^T) \quad (11)$$

Finally, the coefficient matrix Ξ_k at iteration k is updated using:

$$\Xi_k = \Xi_k + \alpha \beta E_k \quad (12)$$

C. RECURSIVE LASSO WITH ADAM OPTIMIZER

For more complex systems or highly nonlinear and highly coupled systems, RLS is unable to produce sparse equations and can overfit. To overcome these shortcomings, recursive LASSO (Least Absolute Shrinkage and Selection Operator) is proposed, which is a LASSO method for streaming data that updates its gain every time new data arrives. In this study, the recursive LASSO method was optimized with the Adam Optimizer and used model selection criteria to determine the appropriate gain threshold at each iteration. Similar to RLS, in Recursive LASSO, the covariance matrix Q that stores the input information is defined as

$$\begin{aligned} Q_k &= \sum_{i=1}^k \lambda^{k-i} \mathbf{x}_i^T \mathbf{x}_i \\ &= \lambda Q_{k-1} + \mathbf{x}_k^T \mathbf{x}_k \end{aligned} \quad (13)$$

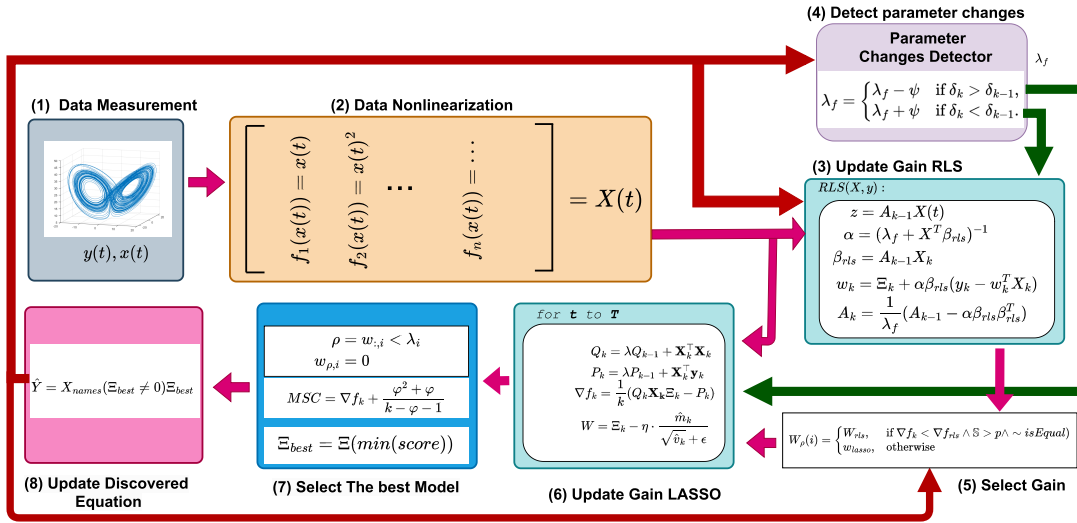


FIGURE 3. Recursive Sparse Identification of Nonlinear Dynamics.

where $Q_k \in \mathbb{R}^{n \times n}$ is a matrix that stores the input information for each iteration, $x_k \in \mathbb{R}^n$ is the current input vector that has been nonlinearized and λ is the forgetting factor gain. While the matrix $P \in \mathbb{R}^n$ that stores the output information is expressed as follows,

$$P_k = \sum_{i=1}^k \lambda^{k-1} \mathbf{x}_i^\top \mathbf{y}_i = \lambda P_{k-1} + \mathbf{x}_k^\top \mathbf{y}_k \quad (14)$$

In Recursive LASSO, gradient calculations are performed to determine which gain should be tuned and the amount of change required. The gradient formula is stated as follows:

$$\nabla f_k = \frac{1}{k} (Q_k \mathbf{x}_k \Xi_k - P_k) \quad (15)$$

Gradient ∇f_k provides information on the magnitude and direction of the gain change. However, if the gradient value is too large, it can cause large oscillations in the parameter adjustment and slow down error convergence. To overcome this, the first-moment gradient is required, which is defined as follows

$$m_k = \beta_1 m_k + (1 - \beta_1) \nabla f_k$$

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^\mu} \quad (16)$$

where β_1 is the first-moment gradient constant and μ is the epoch number. The second-moment gradient is also used to overcome the parameter update, which is too large with the appropriate proportion based on the previous second-moment gradient calculation. The second-moment gradient is defined by the following equation:

$$v_k = \beta_2 \cdot v_k + (1 - \beta_2) (\nabla f_k)^2$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^\mu} \quad (17)$$

where β_2 is the second-moment gradient constant and μ is the epoch number, and the equation for tuning gain W is expressed as follows:

$$W = \Xi_k - \eta \cdot \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} \quad (18)$$

where W denotes a candidate for Ξ . In this study, a hard threshold was used, as in SINDy [15] instead of the soft threshold commonly used in Batch LASSO. The hard threshold is defined as follows,

$$W_\rho(i) = \begin{cases} W_i, & \text{if } |W_i| \geq \rho \\ 0, & \text{if } |W_i| < \rho \end{cases} \quad (19)$$

where W_ρ is a candidate gain for each threshold ρ , and the best W_ρ is selected using the model selection criteria defined in the following equation:

$$\Xi = W_{\rho\text{-best}} \quad (20)$$

D. MODEL SELECTION CRITERIA

A challenge arises when employing only one threshold during regression, as it may result in the elimination of all gain coefficients if the threshold is excessively large during the initialization of the system or when detecting new parameter changes. Conversely, if the threshold is too small, the gain elimination process may become less effective, leaving unnecessary gains. To address this issue, a model selection approach is proposed to determine the most suitable threshold during the process based on sample data. The model selection criteria method utilized in this work is based on the modified Corrected Akaike Information Criterion (AIC), which is expressed as follows:

$$MSC = \nabla f_k + \frac{\varphi^2 + \varphi}{k - \varphi - 1} \quad (21)$$

where φ is the number of nonzero coefficients of gain Ξ_k . Equation (21) is applied at each iteration to select the

best coefficient Ξ_k of current iteration k . The best model gain coefficients are obtained by considering the smallest calculated selection criterion value.

E. PARAMETER CHANGES DETECTOR (PCD)

Parameter changes can occur when the system is running, which can be caused by internal problems in the system (faults) or external disturbances. The identification system method is important for detecting these changes quickly, so that the next steps can be determined immediately. One of the contributions of this study is the improved adaptive capability to detect changes in system parameters and take action, such that the gain can change more quickly to adapt to the latest system conditions. Parameter changes include parameter values or state coefficients that change or add variables to the state. Welford's online variance algorithm statistical method was used to detect changes in the parameters [34], [35] to detect changes in the errors. The PCD algorithm is based on the following lemma and theorem.

The error prediction or modeling error E_k at iteration k is defined as the difference between the actual value and its predicted value, which is expressed as follows:

$$E_k = y_k - \Xi_k^T x_k \quad (22)$$

where k denotes the k th iteration. The error distance δ was used to measure the distance of error E_k from the mean error $\bar{E}_{k-1}$, which is defined as follows:

$$\delta = E_k - \bar{E}_{k-1} \quad (23)$$

mean of error $\bar{E}_k$ at k -th iteration is stated as follows,

$$\bar{E}_k = \bar{E}_{k-1} + \frac{\delta}{k} \quad (24)$$

Lemma 1: Let M be the aggregate of error deviations, defined as the sum of the previous M and the squared deviations from the mean error prediction:

$$M_k = M_{k-1} + \delta(E_k - \bar{E}_k) \quad (25)$$

Then $M \geq 0$ for $k > 1$.

Proof: Aggregate of deviations M can be expressed in the following form:

$$M_k = M_{k-1} + \delta(E_k - \bar{E}_{k-1} - \frac{\delta}{k}) \quad (26)$$

$$= M_{k-1} + (E_k - \bar{E}_{k-1})^2 - \frac{1}{k}(E_k - \bar{E}_{k-1})^2 \quad (27)$$

$$= M_{k-1} + \delta(\delta - \frac{\delta}{k}) \quad (28)$$

It can be seen that $\delta(\delta + \frac{\delta}{k})$ is non-negative since it is the square of real numbers regardless of whether $\delta > 0$ or $\delta < 0$. Therefore, their sum M is non-negative:

$$\delta(\delta + \frac{\delta}{k}) \geq 0 \quad \forall k \Rightarrow M_k = M_{k-1} + \delta(E_k - \bar{E}_k) \geq 0. \quad (29)$$

Definition 1: An error distance δ^2 is said to be **outlier** if:

$$\delta_k^2 > \frac{M_{k-1}k}{(k-1)(k-2)} \quad (30)$$

Theorem 1: If the modeling error has stabilized, such that the error distance δ^2 is not an outlier (i.e., it is within acceptable bounds), then the variance σ_k will decrease towards zero as k approaches infinity.

Proof: The variance σ_k at iteration k is defined as:

$$\sigma_k = \frac{M_k}{k-1} \quad (31)$$

$$\sigma_k = \frac{1}{k-1}(M_{k-1} + \delta^2 - \frac{\delta^2}{k}) \quad (32)$$

Since M_k is nonnegative, σ_k is also nonnegative for $k > 1$. As k increases, R-SINDy with proper matrix coefficients Ξ drives E_k to zero, causing the error distance δ^2 to decrease, and the threshold for the squared distance of the error will also decrease. This implies that the aggregate of squared deviations M_k increases slightly, and k will continue to increase such that the denominator in (31) becomes larger than M_k . Therefore, variance σ_k becomes

$$\sigma_k = \frac{M_k}{k-1} \rightarrow 0 \quad \text{as } k \rightarrow \infty \quad (33)$$

This shows that the variance σ_k approaches zero if δ^2 is not an outlier. ■

Theorem 2: If there is a change in the system's parameters causing a drastic increase in the error such that the squared error distance δ^2 becomes an outlier, the variance will increase beyond its previous value.

Proof: If a system parameter changes and the matrix coefficients Ξ no longer align with the new parameters, this will cause the squared error distance σ_k to increase substantially and become an outlier. Let us start with the current and previous variances,

$$\sigma_k > \sigma_{k-1} \quad (34)$$

$$\frac{M_k}{k-1} > \frac{M_{k-1}}{k-2} \quad (35)$$

Let $Q = k - 1$ then,

$$\frac{M_k}{Q} > \frac{M_{k-1}}{Q-1} \quad (36)$$

multiplying both sides:

$$M_k(Q-1) > M_{k-1}Q \quad (37)$$

Expanding M_k as in (25), then:

$$\frac{M_{k-1}k + \delta_k^2k - \delta_k^2}{k}(Q-1) > M_{k-1}Q \quad (38)$$

Simplifying the expression results in,

$$\delta_k^2(Q(k-1) - (k-1)) > M_{k-1}k \quad (39)$$

rewrite Q as $k - 1$ and factorize:

$$\delta_k^2 > \frac{M_{k-1}k}{(k-1)(k-2)} \quad (40)$$

This proves that when δ_k^2 exceeds the threshold or becomes an outlier, the variance σ_k increases compared to its previous value. This result can be leveraged to detect significant parameter changes and expedite necessary actions to stabilize the modeling error.

The online variance method defined in (22)–(34) is performed by calculating the change in the modeling error with a significant value when a change in the system parameters occurs. This significant change also causes the variance value to be greater than before, so that the PCD algorithm compares the current variance of error σ_k with the previous variance of error σ_{k-1} as stated in the Algorithm 1.

Algorithm 1 Parameter Changes Detector (PCD)

```

1 Input:  $e_k, e_{k_1}, M_{k-1}, \sigma_{k-1}, k$ 
2 Output:  $\lambda$  // forgetting Factor gain
3  $E_k = y - \Xi^T x$  // (22)
4  $\Delta\sigma = E_k - \bar{E}_{k-1}$  // (23)
5  $\bar{E}_k = \bar{E}_{k-1} + \frac{\Delta\sigma}{k}$  // (24)
6  $M_k = M_{k-1} + \Delta\sigma(E_k - \bar{E}_k)$  // (25)
7  $\sigma_k = \frac{M_k}{k-1}$  // (31)
8 if  $\sigma_k \geq \sigma_{k-1}$  &  $\lambda > 0$  then
9    $\lambda = \lambda - \psi$  // Reduce  $\lambda$ 
10 end
11 else
12    $\lambda = \lambda + \psi$  // Increase  $\lambda$ 
13 end
14 Return  $\lambda$ 

```

F. ALGORITHM OF RECURSIVE SPARSE IDENTIFICATION OF NONLINEAR DYNAMICS

The following algorithm is used to process the input and output data into a sparse coefficient matrix Ξ using the RLS and recursive LASSO. The R-SINDy algorithm is described in the Algorithm 2.

Gain selector used to select between the gain w_{rls} and w_{lasso} is expressed using the following equation,

$$W_\rho(i) = \begin{cases} w_{rls}, & \text{if } \nabla f_k < \nabla f_{rls} \wedge \mathbb{S} > p \wedge \sim isEqual \\ w_{lasso}, & \text{otherwise} \end{cases} \quad (41)$$

where $\sim isEqual$ is used to ensure that the current non-zero w_{rls} is not equal to the previous non-zero w_{rls} . $\mathbb{S}$ is the sparsity value expressed in (42), but the calculation is performed for the gain w_{rls} .

G. EVALUATIONS

An evaluation was conducted to measure the effectiveness of the proposed method, R-SINDy, which is based on recursive least-squares assisted LASSO. The proposed method was compared with the Recursive Least Squares (RLS) method to observe its impact on the baseline method.

Algorithm 2 R-SINDy

```

1 Input:  $x_{new}, y, \Xi$ 
  // new data arrive
2 Output:  $\Xi$ 
3  $x_k = \text{nonlinearize}(x_{new})$ 
4  $Q_k = \lambda Q_{k-1} + x_k^T x_k$  // (13)
5  $P_k = \lambda P_{k-1} + x_k^T y_k$  // (14)
6 PCD(); // PCD Algorithm 1
7  $w_{rls} = \text{RLS}()$  // RLS (8) - (12)
  // get RLS gain if meets the
  criteria (41)
8 gainSelector( $w_{rls}, \Xi, P_k, Q_k$ )
9 for  $\mu = 1$  to  $\mathbb{E}$  do //  $\mathbb{E}$  is Total epoch
  // calculate gradient (15)
10  $\nabla f_k = \frac{1}{k}(Q_k x_k \Xi_k - P_k)$ 
  // calculate first moment
  gradient (16)
11  $m_k = \beta_1 m_k + (1 - \beta_1) \nabla f_k$ 
12  $\hat{m}_k = \frac{m_k}{1 - \beta_1^\mu}$ 
  // calculate second moment
  gradient (17)
13  $v_k = \beta_2 \cdot v_k + (1 - \beta_2)(\nabla f_k)^2$ 
14  $\hat{v}_k = \frac{v_k}{1 - \beta_2^\mu}$ 
  // calculate candidate of  $\Xi$  (18)
15  $W = \Xi_k - \eta \cdot \frac{\hat{m}_k}{\sqrt{\hat{v}_k + \epsilon}}$ 
16 for  $j = 1$  to  $J$  do //  $J$  is Number of
  Penalty
17    $w_j = W$ ;
  // Hard Threshold (19)
18    $w_j(|W| < \rho_j) = 0$ 
  // Model Selection Criteria
19    $score_j = \text{MSC}(Q_k, P_k, w_j, t)$ 
20 end
  // Select the best candidate
21  $\rho_{best} = \min(score)$ 
  // take the gain value from the
  best candidate (20)
22  $\Xi = w_{j=\rho_{best}}$ 
23 end
24 Return  $\Xi$ 

```

1) FITTING SCORE

As the result of the recursive process is obtained at each instant, it is necessary to measure the fitting score at each step. The Fitting Score was used to determine the fitting scores produced by both the methods.

2) ADAPTIVITY

The capabilities of both methods were tested against changes in the parameters within the system. The desired system should be able to model parameter changes or disturbances during operation.

3) SPARSITY

This test is conducted to assess the ability of the proposed method to eliminate coefficients that do not match those of the original model. The sparsity equation is stated as follows,

$$\mathbb{S} = \left(\frac{\|\Xi\|_0}{Z} \right) \times 100\% \quad (42)$$

where $\mathbb{S}$ is sparsity percentage, $\|\Xi\|_0$ is number of zero element of gain Ξ , and Z is the total element of Ξ .

IV. RESULTS AND DISCUSSIONS

This section discusses the results of recursive sparse identification of nonlinear dynamics (R-SINDy) using the proposed method. The results were obtained using computer simulations to simulate streaming data. The system models used were the Lorenz Chaotic System and a 3-DoF manipulator robot.

A. FUNCTION LIBRARIES

The nonlinear function library used to generate nonlinear candidates is divided into two parts because there are differences in the nonlinearization mechanisms in the Lorenz system and the robot manipulator. In robot manipulators, the resulting equations are expected to follow the rules that satisfy the robot manipulator equations, as stated in [29]. The nonlinear functions are presented in Table 1 as follows:

TABLE 1. Library function.

No	Lorenz System	N	Robot Manipulator	N
1	@(i) i	3	@(z) z	3
2	@(i,j) i · j	3	@(p) C(p)	3
3	@(i,j) i ²	3	@(u) sign(u) · u ²	3
4	@(i) C(i)	3	@(p,r) C(p + r)	9
5	@(i) S(i)	3	@(p,z) S(p) · z	9
6			@(p,z) C(p) · z	9
7			@(p,u,v) S(p) · u · v	27
8			@(p,r,z) C(p + r) · z	27
9			@(p,r,z) C(2 * p + r) · z	27
10			@(p,r,u,v) S(p + r) · u · v	81
11			@(p,r,u,v) S(2p + r) · u · v	81
12			@(u) u	3
Total Variable $\sum N$		15		282

where $C(\cdot)$ and $S(\cdot)$ are the abbreviations for $\cos(\cdot)$ and $\sin(\cdot)$ respectively. In the Lorenz system, trigonometric library functions are used to test the reliability of the proposed method in overcoming the effects of various library functions. Thus, the system can still have appropriate and sparse equations. The parameters of the R-SINDy used by both systems are listed in Table 2.

B. LORENZ SYSTEM

In the experiment using the Lorenz system, the model equation used is shown in (1). Using the model parameter values listed in Table 3,

The results of online identification using the proposed method with a total of 1,000 iterations of data are shown

TABLE 2. The parameter of R-SINDy.

No	Parameter	Result
1	First Moment Gain (β_1)	0.1
2	Second Moment Gain (β_2)	0.99
3	Learning Rate (η)	1e-3
4	Forgetting Gain Adjustment	0.05
5	Number of epoch	10
6	Variation of Lambda(logspace)	-5 to 0 (10)
7	RLS Sparsity Threshold	0.3

TABLE 3. The model parameter of Lorenz system.

No	Symbol	Value
1	ϱ	10
2	ρ	28
3	β	-2.6

TABLE 4. Identification results summary of Lorenz system.

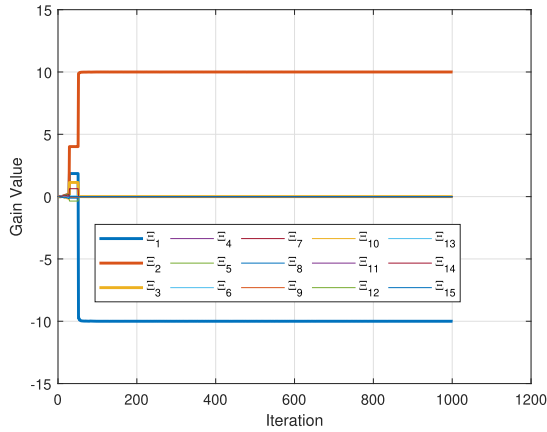
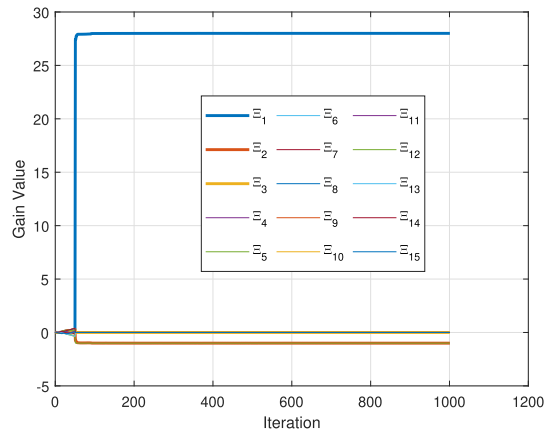
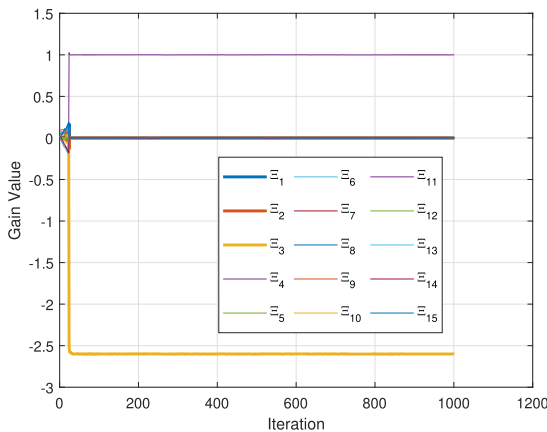
No	Parameter	Result		
1	Mean of Regression Time	0.0024 s		
2	Number of Gains $\sum N$	15		
3	Number of Nonzero Gains	Nonzero (sparsity)		
	- State 1	2 (86.67 %)		
	- State 2	3 (80.0 %)		
	- State 3	4 (73.33 %)		
4	Fitness Scores	DTr		
	- State 1	100 %		
	- State 2	100 %		
	- State 3	100 %		
5	RMSE			
	- Joint 1	0.82	1.05	1.16
	- Joint 2	1.565	1.567	2.16
	- Joint 3	1.15	1.46	1.92

in Table 4.

$$\begin{aligned} \dot{x}_L &= 9.999y_L - 10.001x_L \\ \dot{y}_L &= 28.000x_L - 0.9997y_L - 1.000x_L z_L \\ \dot{z}_L &= -2.601z_L + 0.999x_L y_L + \\ &\quad - 0.0013631C(x_L) + -0.0013834C(z_L); \end{aligned} \quad (43)$$

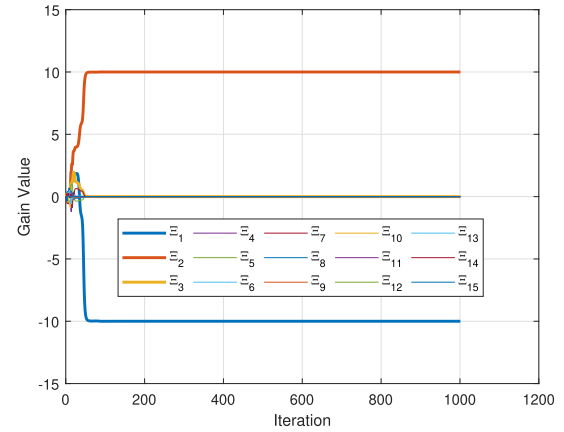
In Table 4 the regression time is calculated at each iteration, whereas the equation printing time refers to the time taken to print the obtained gains into mathematical equations. Regarding sparsity, the proposed method has proven capable of eliminating many variables that have a minimal impact on the system's dynamics and producing mathematical equations that closely match the actual equations of the Lorenz system. The process of gain change is shown in Figs. 4, 5, and 6.

From Fig. 4 it can be seen the change in gain during the regression process using the proposed method, the final gain is obtained at iteration 53, where only two gains remain that match the identified dynamic model. This also occurs in Figs. 5 and 6 where the gain changes suddenly to the gain value that matches the actual system. This is due to the assistance of the RLS. RLS in a simple system has a superior ability to model the system; therefore, when the RLS

FIGURE 4. Gains of Lorenz using proposed method on State $\dot{x}_1$.FIGURE 5. Gains of Lorenz using proposed method on State $\dot{y}_1$.FIGURE 6. Gains of Lorenz using proposed method on State $\dot{z}_1$.

criteria are met, the RLS gain is immediately used as a gain by R-SINDy, while the change in RLS gain can be seen in Fig. 7.

The differences between Fig. 4 and Fig. 7 is in the process towards the final gain, where in Fig. 7 the gain value slowly approaches the final gain value, while in Fig. 4 the gain value instantly reaches the final gain value while the other values are below 5. In Recursive LASSO with Adam Optimization,

FIGURE 7. Gains of Lorenz using RLS on State $\dot{x}_1$.

the process to reach the final gain can be performed, but it takes longer. However, for sparsity, Recursive LASSO has more reliable capabilities than RLS in complex systems. Therefore, RLS is needed as an assistant for LASSO to help the gain become faster and more accurate but under the condition that RLS must have produced sparse gain.

C. LORENZ SYSTEM WITH PARAMETER CHANGES

In this scenario, the proposed method was tested using a system that experienced parameter changes during the regression process. Parameter changes in a system can occur because of disturbance factors or system faults, and it is important for the online regression method to detect these parameter changes. It should be noted that the system parameter changes in Lorenz were used only to test the ability of the proposed method to detect system changes. The initial system equation is given by (1). Subsequently, in the 1401-st iteration, the system model parameter changes as follows:

$$\begin{aligned}\dot{x}_L &= \sigma(y_L - x_L) + 1.02 z_L \\ \dot{y}_L &= x_L(\rho - z_L) - y_L \\ \dot{z}_L &= x_L y_L - \beta z_L\end{aligned}\quad (44)$$

where in (44), it can be observed that there is an additional parameter $1.02 z_L$. The results of this test were carried out with 1,900 data points, and the change in variance when detecting a change in error can be seen in Figure 8. From the Figure, it can be seen that the initial error variance value has a very large value but then quickly reaches a value of approximately 725.6 and continues to decrease slowly as iteration k increases. However, at 1403 seconds, the error variance changes its value to be greater than the previous value which is then used to adjust the forgetting factor λ , and the result is that the error variance value decreases slowly again at 1404 seconds and so on. These results indicate that the proposed PCD algorithm can detect disturbances that cause higher regression errors and can be used to take action so that R-SINDy can regress faster to obtain the right gain.

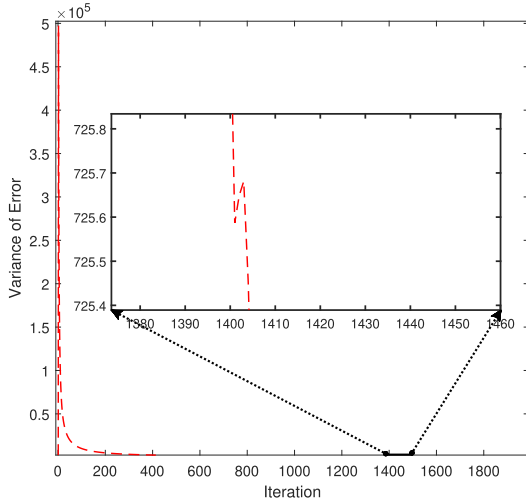


FIGURE 8. Error variance changes during regression process.

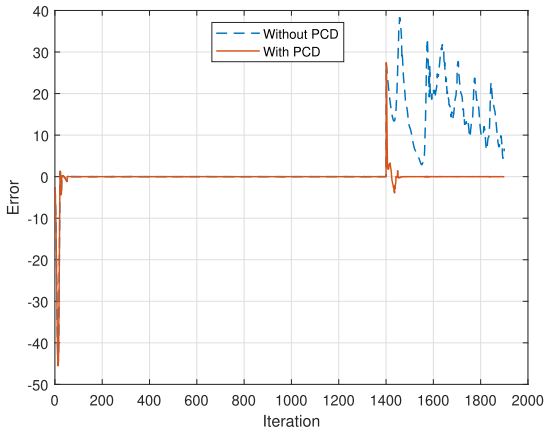


FIGURE 9. Error during regression process.

The error comparison during the regression process is presented in Fig. 9, in the error without PCD when there is a sudden increase in parameter changes which then oscillates slowly towards zero, this shows that without PCD it requires much more data to converge towards error. While using PCD shows the ability to converge towards zero faster owing to forgetting factors that change to be smaller so that old data can be slowly ignored and new data can be used to obtain a gain that is more in accordance with current conditions.

The gain changes are shown in Figs. 10 and 11. The figures show that when using RLS without PCD, when there was a change in the parameters in the system, all the gains changed and then returned to the original values very slowly.

The gain with the PCD is shown in Fig. 11 which shows the ability to obtain faster and more precise gain according to the change in gain that should be $1.02z_L$. The results of the equation for the state $\dot{x}_l$ obtained using PCD are stated in (45).

$$\dot{x}_L = 9.9997y_L - 9.999x_L + 1.02z_L \quad (45)$$

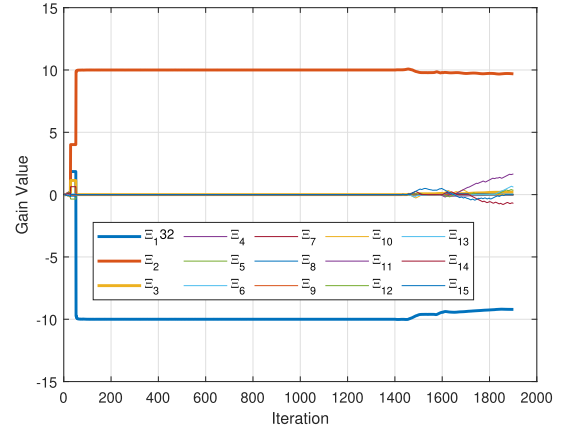


FIGURE 10. Gains changes during the regression process without PCD.

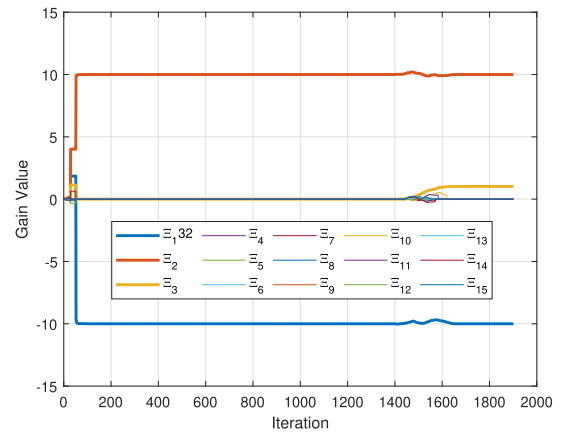


FIGURE 11. Gains changes during the regression process using PCD.

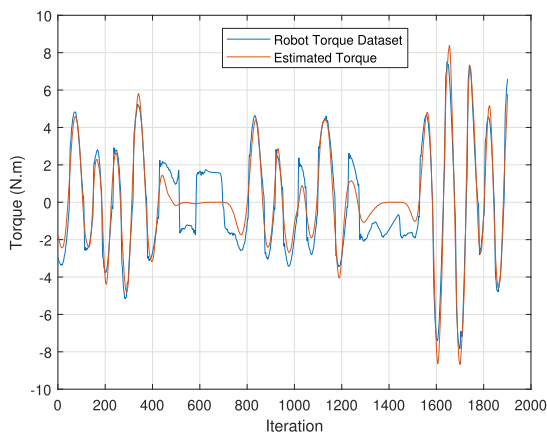
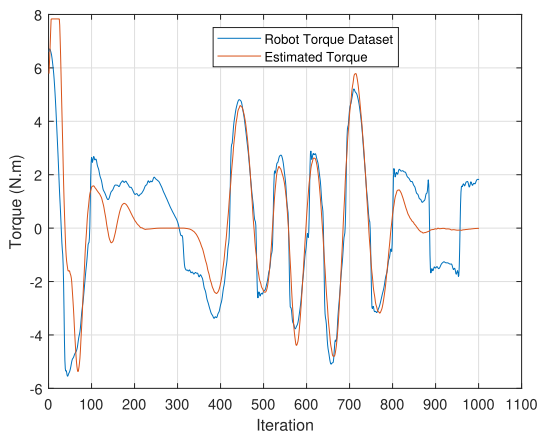
D. 3-DoF KUKA ROBOT MANIPULATOR

Table 1 lists the library functions used in robot manipulator experiment. These library functions were then processed using the Dynamic Expression Nonlinearization (DEx-N) Algorithm, which produced 282 nonlinear candidates listed on Table 1. Testing was performed using the dataset obtained in [31]. The results of the online identification are shown in Table 5.

In Table 5, the number of gains on joint 1 is 35 nonzero gains, or has a sparsity of 87.5 %. With 35 gains, the accuracy for training data estimation is 92.02 %, and on the test data, which is 1000 data after training data, shows an accuracy result of 83.22 %, and testing for all data, namely 18 thousand data where the training data shows a fitting score accuracy of 85.18 %. This shows that sparsity does not reduce the overall accuracy but is also able to reduce overfitting. The fitting results are shown in Figure 12, which indicate that the fitting results coincide with those of the torque dataset. However, there is still a fairly large error in the range of 0, at which point there is a backlash gear, which is a phenomenon where there is a large friction that inhibits the movement of the robot arm until there is a torque large enough for the robot arm to

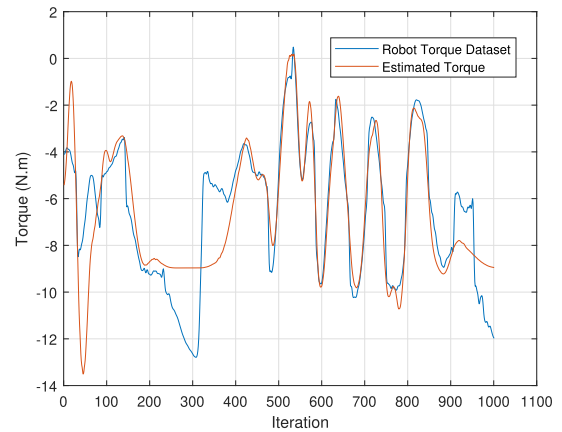
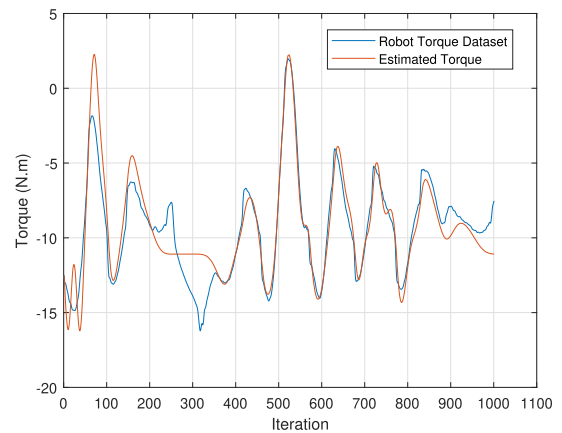
TABLE 5. Online identification results summary of KUKA 3-DOF robot manipulator.

No	Parameter	Result		
1	Mean of Regression Time	0.0185 s		
2	Number of Gains $\sum N$	282		
3	Number of Nonzero Gains	Nonzero (sparsity)		
	- Joint 1	35 (87.59 %)		
	- Joint 2	77 (72.70 %)		
	- Joint 3	50 (82.27 %)		
4	Fitness Scores	DTr	Next-1000	All-18,000
	- Joint 1	92.02 %	83.22 %	85.18 %
	- Joint 2	74.74 %	67.33 %	58.71 %
	- Joint 3	85.74 %	81.73 %	69.16 %
5	RMSE			
	- Joint 1	0.82	1.05	1.16
	- Joint 2	1.565	1.567	2.16
	- Joint 3	1.15	1.46	1.92

**FIGURE 12.** The estimated torque of Joint 1 on training data.**FIGURE 13.** The estimated torque of Joint 1 on next 1000 data.

move. A special nonlinear function was required to model the backlash effect.

The fitting results obtained using 1000 forward data points after the training data are shown in Fig. 13. The fitting results have the same weakness as the training data, namely that they cannot estimate well at torque positions around zero due to the backlash effect. In addition to the backlash position, the obtained gain can accurately estimate the torque.

**FIGURE 14.** The estimated torque of Joint 2 on next 1000 data.**FIGURE 15.** The estimated torque of Joint 3 on next 1000 data.

The results of fitting 1000 forward data points for joints 2 and 3 are shown in Figs. 14 and 15. The fitting results were quite good, except for the position owing to backlash; at joint 2, the backlash position was 9 N. At joints 2 and 3, the movement is influenced by gravity, so the torque required to move the motor has a different character from that at joint 1, where 0 Nm is a neutral torque that does not move the robot. At joints 2 and 3, when the torque is 0 N.m, it can move the robot towards the gravitational pull, and at a torque of approximately -8, it has a pushing force that can counteract the gravitational pull.

Changes in the forgetting factor gain λ for the three joints are shown in Fig. 16. The forgetting factor gain value remains constant for joints 1 and 3 at value of 1.0, while joint 2 experiences a change until the forgetting factor gain reaches a value of 0.95. This indicates that some gains must be adjusted to improve regression results. The effect of this change in the forgetting factor on gain Ξ can be observed in joint 2, as illustrated in Fig. 17.

On Fig. 18, it can be observed the very significant changes in the RLS gain during the regression process on joint 2. In addition, all 282 gains have their own values or full density, which causes the RLS to become overfitted to all data (18,000) with a fitting score below 0 %.

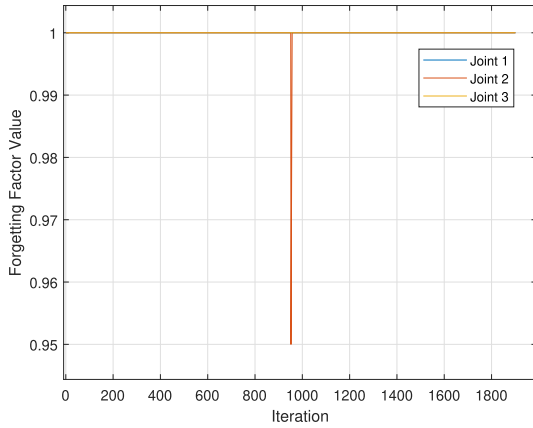


FIGURE 16. Forgetting factor changes during the regression process on.

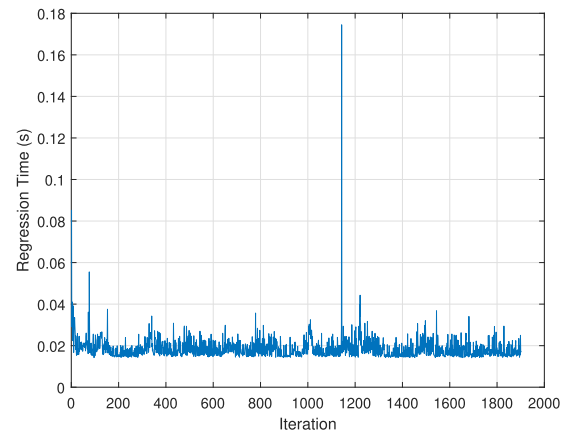


FIGURE 19. Regression time during the regression process on Joint 1.

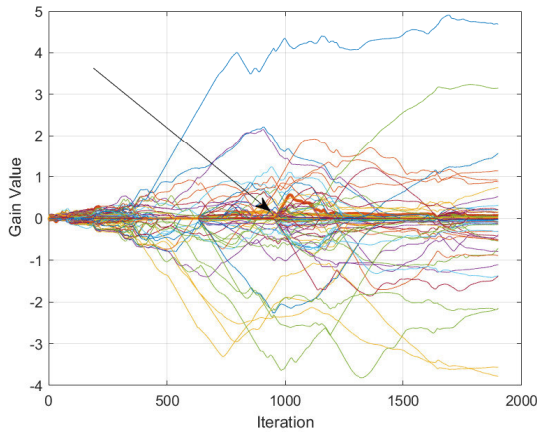


FIGURE 17. The gain changes of the R-SINDy during the regression process on Joint 2.

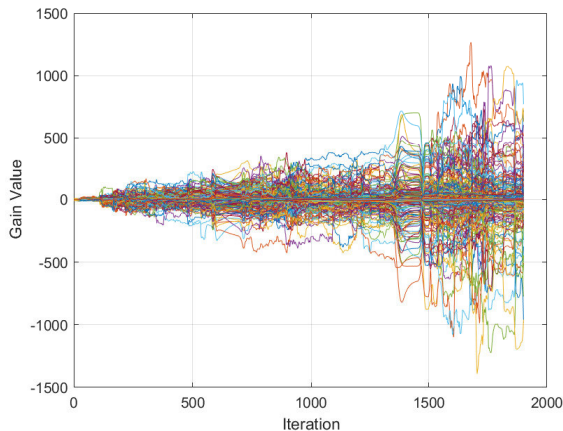


FIGURE 18. The gain changes of the RLS during the regression process on Joint 2.

The time required for each regression is shown in Fig. 19, the average time required is 0.0185 s, with the number of regressors of 282. An improvement over the average Lorenz regression time using only 15 regressors. This time is sufficiently fast to be used for online identification and can be used in systems with a minimum data sampling time of 50ms.

TABLE 6. Online identification Results summary using other method.

No	Parameter	RLS	Sparse RLS [26]	L_0 -RLS [25]
1	Number of Nonzero Gains			
	- Joint 1	282 (0 %)	240(14.89 %)	66(76.60%)
	- Joint 2	282 (0 %)	259(8.16 %)	44(84.40%)
	- Joint 3	282 (0 %)	274(14.89 %)	68(75.89%)
2	Fitness score on Train Data			
	- Joint 1	95.62 %	79.94 %	34.6357 %
	- Joint 2	83.80 %	57.84 %	-499.01 %
	- Joint 3	89.74 %	73.73	-1070.78 %
3	Fitness score on Test Data			
	- Joint 1	-209577 %	52.85 %	23.59 %
	- Joint 2	-695202 %	1.23 %	-513.25 %
	- Joint 3	-23399 %	32.55 %	-786.86%

To evaluate the effectiveness of the proposed method, it was compared with other methods, namely, RLS, Sparse RLS [26], and L_0 -RLS [25]. The results obtained using these methods are presented in Table 6.

From the results shown in Table 5 and 6, which compares the R-SINDy, RLS, Sparse RLS, and L_0 -RLS methods, it is evident that these approaches can successfully identify the training data except L_0 -RLS in joints 2 and 3. However, when applied to test data, a significant drop in accuracy was observed, particularly for RLS and L_0 -RLS. In contrast, Sparse RLS maintained relatively good accuracy with a sparsity of 14.89%. Based on these findings, the RLS, Sparse RLS, and L_0 -RLS methods still exhibit overfitting when applied to nonlinear and highly coupled systems, such as robot manipulators. The R-SINDy method, however, appears to handle overfitting more effectively in robot manipulator with accuracy on test data above 58 % for all joints.

The equation that has been discovered using R-SINDy can be seen in Table 7 because the equation is still too long, this equation is truncated and the complete equation can be seen in the url provided. The time required to convert the gain into the D, H, and G equations is 27.3984 s, so the printing mechanism

TABLE 7. Discovered equation of KUKA Robot.

$$\begin{aligned}
D &= \begin{bmatrix} 0.1618C(q_1) + \dots & 0.6626C(q_1) + \dots & 0.1633S(q_2) + \dots \\ 0.686C(q_3) + \dots & 0.1302C(q_3) - \dots & 0.2785C(q_3) + \dots \\ 0.1586C(q_1)^2 C(q_3) + \dots & -0.05783C(q_1) & 0.1922C(q_1)^3 + \dots \end{bmatrix} \begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \\ \ddot{q}_3 \end{bmatrix} \\
H &= \begin{bmatrix} 3.307 * \dot{q}_1 - 0.07304 * \dot{q}_2 + \dots \\ 3.074 * \dot{q}_2^2 * S(q_2) - 2.405 * \dot{q}_2^2 * S(q_1) + \dots \\ 1.339 * \dot{q}_3 + 0.4256 * \dot{q}_1^2 * S(q_1) + \dots \end{bmatrix} \\
G &= \begin{bmatrix} 0.0005553 * C(q_2) + 0.0004653 * C(q_3) + \dots \\ 2.219 * S(q_1) * S(q_3) - 2.219 * C(q_1) * C(q_3) + \dots \\ 1.657 * S(q_1) * S(q_3) - 1.657 * C(q_1) * C(q_3) + \dots \end{bmatrix}
\end{aligned}$$

$C(\cdot) = \cos(\cdot)$, $S(\cdot) = \sin(\cdot)$
 The complete equation is available on <https://github.com/swadexi/R-SINDy>

for this equation needs to be performed at the appropriate moment, that is, when the identification system has stabilized.

V. CONCLUSION

This paper proposes a Recursive Sparse Identification of Nonlinear Dynamics (R-SINDy) method, based on RLS-Assisted LASSO with a dynamic Forgetting Factor, for online equation discovery. The proposed method processes each new single row of data to update the gain Ξ , and this process takes relatively fast time, which is 0.0013 seconds for 15 inputs in the Lorenz system and 0.018 seconds in a 3-DoF robot manipulator with the number of input vectors that have been nonlinearized to 282 inputs.

The accuracy results obtained reached 99.99 % for the dynamics of the Lorenz system and were also able to find new gains quickly when there was a change in the parameters. The accuracy of the robot dynamics identification can reach 92 % and is able to estimate several thousand data in the future or unknown data, even though only 1,900 training data are used. From the results of the identification of the dynamics of these two nonlinear systems, it can be concluded that the proposed method can be used for the discovery of nonlinear system equations and, with further research, can be used for various purposes, such as fault identification, adaptive systems, or monitoring system dynamics.

RLS-assisted LASSO has a positive impact, where RLS performs exceptionally well on simple systems, while Adam Optimized LASSO can work more effectively on more complex nonlinear systems. Relying solely on RLS can result in overfitting and full density, which are less suitable for equation discovery because of the excessive number of gains that need to be displayed.

The role of the Parameter Change Detector is crucial for accelerating the convergence of the error to zero. By adjusting the forgetting factor, the old data that are no longer relevant to the new data are eliminated, allowing the gain Ξ to be updated to match the current system conditions.

The resulting Lorenz system equation shows that the results are very close to the identified Lorenz equation, with a small error of approximately 0.005 for each Lorenz gain. In the robot manipulator dynamics equation, the resulting equation can be expressed in the form of a robotic equation consisting of inertia, gravity, and

coriolis-centrifugal components. This can be achieved using dynamic expression nonlinearisation. However, the Dynamic Expression algorithm still needs to be improved in online system identification, because the process carried out still takes quite a long time.

Based on the comparison with other methods, the R-SINDy method demonstrates relatively higher accuracy in both training and test data evaluations. The accuracy of R-SINDy was the lowest at 58.71%, while the other methods showed accuracy below 50%. This indicates that the R-SINDy method can perform well even in nonlinear and highly coupled systems.

VI. FUTURE WORKS

This research primarily focuses on achieving high accuracy and sparsity levels. However, future studies should aim to improve computational speed by eliminating the use of Recursive Least Squares (RLS) and enhancing the capabilities of the recursive LASSO. The time required to print the equation is still too long, and a more efficient technique needs to be used as a real-time monitoring system so that it can be processed faster.

APPENDIX

ABBREVIATIONS

DEx-N:	Dynamic Expression Nonlinearization.
DeepOnet:	Deepoperator Neural Networks.
DoF:	Degree of Freedoms.
LASSO:	Least Absolute Shrinkage and Selection Operator.
PCD:	Parameter Change Detector.
RLS:	Recursive Least-Square.
R-SINDy:	Recursive Sparse Identification of Nonlinear Dynamics.

ACKNOWLEDGMENT

The authors would like to express their gratitude to Artificial Intelligence language tools such as Google Translate, Paperpal, and ChatGPT, which were used for proofreading and polishing the English in this article.

REFERENCES

- [1] J. Slotine and L. Weiping, *Applied Nonlinear Dynamics*. Upper Saddle River, NJ, USA: Prentice-Hall, 1991.
- [2] B. L. Stevens and F. L. Lewis, *Aircraft Control and Simulation*. Hoboken, NJ, USA: Wiley, 1992.
- [3] O. Sergiyenko, V. Tyrsa, A. Zhirabok, and A. Zuev, "Sensor fault identification in linear and nonlinear dynamic systems via sliding mode observers," *IEEE Sensors J.*, vol. 22, no. 11, pp. 10173–10182, Jun. 2022.
- [4] J. Schoukens, J. Nemeth, G. Vandersteen, R. Pintelon, and P. Crama, "Linearization of nonlinear dynamic systems," in *Proc. IEEE Instrum. Meas. Technol. Conf.*, vol. 2, May 2003, pp. 1524–1527.
- [5] V. Sharma and Y. Zhao, "Dynamic optimal linearization of nonlinear systems," in *Proc. Amer. Control Conf.*, Jun. 1993, pp. 1196–1197.
- [6] K. Ogata, *Modern Control Engineering*, 4th ed., Upper Saddle River, NJ, USA: Prentice-Hall, 2010.
- [7] F. Fahmizal, M. Arrofiq, and A. Mayub, "Identifikasi pemodelan matematis robot wall following," *Jurnal Nasional Teknik Elektro dan Teknologi Informasi*, vol. 7, no. 1, pp. 79–88, Mar. 2018.

- [8] M. S. Esteves, T. P. Perdicoulis, and P. L. D. Santos, "System identification methods for identification of state models," in *CONTROLO'2014—Proceedings of the 11th Portuguese Conference on Automatic Control* (Lecture Notes in Electrical Engineering), vol. 321, Porto, Portugal. Cham, Switzerland: Springer, 2015, pp. 417–427. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-10380-8_40
- [9] L. Ljung, C. Andersson, K. Tiels, and T. B. Schön, "Deep learning and system identification," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 1175–1181, 2020.
- [10] H. A. Nielsen and H. Madsen, "Modelling the heat consumption in district heating systems using a grey-box approach," *Energy Buildings*, vol. 38, no. 1, pp. 63–71, Jan. 2006.
- [11] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Mach. Intell.*, vol. 3, no. 3, pp. 218–229, Mar. 2021.
- [12] J. He, S. Kushwaha, J. Park, S. Koric, D. Abueidda, and I. Jasiuk, "Sequential deep operator networks (S-DeepONet) for predicting full-field solutions under time-dependent loads," *Eng. Appl. Artif. Intell.*, vol. 127, Jan. 2024, Art. no. 107258.
- [13] J. He, S. Koric, D. Abueidda, A. Najafi, and I. Jasiuk, "Geom-DeepONet: A point-cloud-based deep operator network for field predictions on 3D parameterized geometries," *Comput. Methods Appl. Mech. Eng.*, vol. 429, Sep. 2024, Art. no. 117130.
- [14] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Nat. Acad. Sci. USA*, vol. 113, no. 15, pp. 3932–3937, Apr. 2016.
- [15] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Sparse identification of nonlinear dynamics with control (SINDYc)," *IFAC-PapersOnLine*, vol. 49, no. 18, pp. 710–715, 2016.
- [16] N. M. Mangan, J. N. Kutz, S. L. Brunton, and J. L. Proctor, "Model selection for dynamical systems via sparse regression and information criteria," *Proc. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 473, no. 2204, Aug. 2017, Art. no. 20170009.
- [17] U. Fasel, J. N. Kutz, B. W. Brunton, and S. L. Brunton, "Ensemble-SINDy: Robust sparse model discovery in the low-data, high-noise limit, with active learning and control," *Proc. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 478, no. 2260, Apr. 2022, Art. no. 20210904.
- [18] E. Kaiser, J. N. Kutz, and S. L. Brunton, "Sparse identification of nonlinear dynamics for model predictive control in the low-data limit," *Proc. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 474, no. 2219, Nov. 2018, Art. no. 20180335.
- [19] R. L. Plackett, "Some theorems in least squares," *Biometrika*, vol. 37, no. 1, p. 149, Jun. 1950. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/15420260/>
- [20] S. Elliott, *Optimal and Adaptive Digital Filters*. New York, NY, USA: Academic, 2001, pp. 49–102.
- [21] S. A. U. Islam and D. S. Bernstein, "Recursive least squares for real-time implementation [lecture notes]," *IEEE Control Syst. Mag.*, vol. 39, no. 3, pp. 82–85, Jun. 2019.
- [22] M. A. Syed and V. J. Mathews, "Lattice algorithms for recursive least squares adaptive second-order Volterra filtering," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 41, no. 3, pp. 202–214, Mar. 1994.
- [23] P. S. R. Diniz, "Adaptive lattice-based RLS algorithms," in *Adaptive Filtering*. Boston, MA, USA: Springer, 2013.
- [24] S. Haykin, *Adaptive Filter Theory*, 5th ed., London, U.K.: Pearson, 2014.
- [25] H. Yazdanpanah and P. S. R. Diniz, "Recursive least-squares algorithms for sparse system modeling," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Mar. 2017, pp. 3879–3883.
- [26] B. Babadi, N. Kalouptsidis, and V. Tarokh, "SPARLS: The sparse RLS algorithm," *IEEE Trans. Signal Process.*, vol. 58, no. 8, pp. 4013–4025, Aug. 2010.
- [27] X. Hong, J. Gao, and S. Chen, "Zero-attracting recursive least squares algorithms," *IEEE Trans. Veh. Technol.*, vol. 66, no. 1, pp. 213–221, Jan. 2017.
- [28] X. Ai, J. Zhao, H. Zhang, and Y. Sun, "Sparse sliding-window kernel recursive least-squares channel prediction for fast time-varying MIMO systems," *Sensors*, vol. 22, no. 16, p. 6248, Aug. 2022.
- [29] S. Istiqphara, O. Wahyunggoro, and A. I. Cahyadi, "Enhanced equation discovery of 3-DoF robotic manipulator dynamics using LASSO model selection criteria with variable segregation algorithm," *IEEE Access*, vol. 12, pp. 20574–20590, 2024.
- [30] E. N. Lorenz, "Deterministic nonperiodic flow," *J. Atmos. Sci.*, vol. 20, no. 2, pp. 130–141, Mar. 1963.
- [31] J. Weigand, J. Götz, J. Ulmen, and M. Ruskowski, "Dataset and baseline for an industrial robot identification benchmark," in *Proc. 6th, Ed., Workshop Nonlinear Syst. Identificat. Benchmarks*, 2022, p. 23. [Online]. Available: <https://nbn-resolving.de/urn:nbn:de:hbz:386-kluedo-72843>
- [32] E. C. Ifeachor and B. W. Jervis, *Digital Signal Processing: A Practical Approach*, 2nd ed., Upper Saddle River, NJ, USA: Prentice-Hall, 2002.
- [33] N. T. Nguyen, T. N. T. Nguyen, H. N. Tong, H. V. A. Truong, and D. T. Tran, "Dynamic parameter identification based on the least squares method for a 6-DOF manipulator," in *Proc. Int. Conf. Syst. Sci. Eng. (ICSSE)*, Jul. 2023, pp. 301–305.
- [34] D. E. Knuth, *The Art of Computer Programming Seminumerical Algorithms*, vol. 2. Reading, MA, USA: Addison-Wesley, 1997.
- [35] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, p. 419, Aug. 1962.



SWADEXI ISTIQPHARA (Graduate Student Member, IEEE) was born in Lampung, Indonesia, in 1989. He received the bachelor's degree in electrical engineering from the University of Lampung, in 2012, and the M.S. degree in electrical engineering from the Institut Teknologi Sepuluh Nopember, Indonesia, in 2015. He is currently pursuing the Ph.D. degree in electrical engineering with Universitas Gadjah Mada, Indonesia. He is an Assistant Professor with the Electrical Engineering Department, Institut Teknologi Sumatera, Indonesia. His research interests include system identification, control systems, robotics, and automation.



OYAS WAHYUNGGORO (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Universitas Gadjah Mada, Indonesia, and the Ph.D. degree in electrical and electronic engineering from Universiti Teknologi PETRONAS, Malaysia, in 2011. He is currently a Professor of heuristic control with the Electrical and Information Engineering Department, Universitas Gadjah Mada, Indonesia. His research interests include evolutionary algorithms, fuzzy logic, neural networks, control systems, instrumentation, and battery management systems.



ADHA IMAM CAHYADI received the B.S. degree in electrical engineering from Universitas Gadjah Mada, Indonesia, in 2006, the M.S. degree in control engineering from the King Mongkut's Institute of Technology Ladkrabang, Thailand, in 2008, and the Ph.D. degree from the Unified Graduate School of Engineering, Tokai University, Japan, in 2011. He is currently a Lecturer with the Electrical and Information Engineering Department, Universitas Gadjah Mada. His research interests include multi-agent control systems, control for systems with delays, and control for nonlinear dynamical systems and teleoperation systems.